Week 03 SQL

```
{
  "nbformat": 4,
  "nbformat_minor": 0,
  "metadata": {
    "colab": {
      "provenance": []
    },
    "kernelspec": {
      "name": "python3",
      "display_name": "Python 3"
    },
    "language_info": {
      "name": "python"
    }
  },
  "cells": [
    {
      "cell_type": "code",
      "source": [
        "import pandas as pd, numpy as np, os\n",
        "os.makedirs(\"week-03\", exist_ok=True)\n",
        "\n",
        "def find(df, *words):\n",
        "    for w in words:\n",
        "        for c in df.columns:\n",
        "            if w in c.lower():\n",
        "                return c\n",
        "\n",
        "sales = pd.read_csv(\"/content/sales_features.csv\")\n",
        "inv   = pd.read_csv(\"/content/inventory_features.csv\")\n",
        "led   = pd.read_csv(\"/content/stock_ledger_clean.csv\")\n",
        "\n",
        "s_pid, s_br, s_qty, s_date = (find(sales,\"product_id\"), find(sales,\"branch_id\"),\n",
        "                              find(sales,\"qty\",\"quantity\"), find(sales,\"order_date\",\"date\"))\
n",
        "i_pid, i_br, i_qty = find(inv,\"product_id\"), find(inv,\"branch_id\"), find(inv,\"on_hand\",\"qty\",
\"quantity\",\"stock\")\n",
        "print(\"CHECK:\", s_pid, s_br, s_qty, s_date, \"|\", i_pid, i_br, i_qty)\n",
        "\n",
        "# A. Overstock / understock (months of cover)\n",
        "sales[s_date] = pd.to_datetime(sales[s_date], errors=\"coerce\")\n",
        "months = max((sales[s_date].max() - sales[s_date].min()).days / 30, 1)\n",
        "demand = (sales.groupby([s_pid, s_br])[s_qty].sum() / months).reset_index()\n",
        "demand.columns = [i_pid, i_br, \"avg_monthly_demand\"]\n",
        "\n",
        "cover = inv.merge(demand, on=[i_pid, i_br], how=\"left\")\n",
        "cover[\"months_of_cover\"] = cover[i_qty] / cover[\"avg_monthly_demand\"].replace(0, np.nan)\n",
        "cover[\"status\"] = pd.cut(cover[\"months_of_cover\"], [-np.inf, 1, 6, np.inf],\n",
        "                         labels=[\"Understock (<1 month)\", \"Healthy (1-6)\", \"Overstock (6+
months)\"]).astype(str)\n",
        "cover.loc[cover[\"months_of_cover\"].isna(), \"status\"] = \"No demand\"\n",
        "print(cover[\"status\"].value_counts())\n",
        "print(cover.groupby(\"status\")[\"stock_value\"].sum().round(0))\n",
        "cover.to_csv(\"week-03/stock_status.csv\", index=False)\n",
        "\n",
        "# B. Branch throughput (stock ledger)\n",
        "led[\"movement_date\"] = pd.to_datetime(led[\"movement_date\"], errors=\"coerce\")\n",
        "led[\"month\"] = led[\"movement_date\"].dt.to_period(\"M\").astype(str)\n",
        "thru = led.pivot_table(index=[\"branch_id\",\"month\"], columns=\"movement_type\",\n",
        "                       values=\"quantity\", aggfunc=\"sum\", fill_value=0).reset_index()\n",
        "thru.to_csv(\"week-03/branch_throughput.csv\", index=False)\n",
        "\n",
        "# C. Branch performance score\n",
        "inv[\"dead_value\"] = np.where(inv[\"days_since_last_movement\"] > 180, inv[\"stock_value\"], 0)\n",
        "g = inv.groupby(i_br).agg(stock_value=(\"stock_value\",\"sum\"), dead_value=(\"dead_value\",\"sum\
"))\n",
        "g[\"dead_pct\"] = g[\"dead_value\"] / g[\"stock_value\"]\n",
        "sc = g.join(sales.groupby(s_br)[\"line_revenue\"].sum().rename(\"revenue\"))\n",
        "sc[\"revenue_to_stock\"] = sc[\"revenue\"] / sc[\"stock_value\"]\n",
```

```
        "\n",
        "# Safe normalization that handles constant series without dividing by zero\n",
        "norm = lambda s: (s - s.min()) / (s.max() - s.min()) if (s.max() - s.min()) != 0 else pd.Series(0.0,
index=s.index)\n",
        "\n",
        "sc[\"score\"] = (0.4*norm(sc[\"revenue\"]) + 0.3*norm(sc[\"revenue_to_stock\"]) + 0.3*(1 - norm(sc[\
"dead_pct\"]))).round(3)\n",
        "sc = sc.sort_values(\"score\", ascending=False)\n",
        "print(sc.round(3))\n",
        "sc.to_csv(\"week-03/branch_scores.csv\")"
      ],
      "metadata": {
        "colab": {
          "base_uri": "https://localhost:8080/"
        },
        "id": "n-7EPinNupjV",
        "outputId": "6e6ab682-f50b-4abc-f1b1-54dff92185da"
      },
      "execution_count": 14,
      "outputs": [
        {
          "output_type": "stream",
          "name": "stdout",
          "text": [
            "CHECK: product_id branch_id quantity order_date | product_id branch_id opening_stock\n",
            "status\n",
            "Healthy (1-6)            155\n",
            "Understock (<1 month)     25\n",
            "Name: count, dtype: int64\n",
            "status\n",
            "Healthy (1-6)            443404250\n",
            "Understock (<1 month)     34967320\n",
            "Name: stock_value, dtype: int64\n",
            "           stock_value  dead_value  dead_pct     revenue  revenue_to_stock  \\\n",
            "branch_id                                                                    \n",
            "CHN001        83129460           0       0.0  5192125100            62.458   \n",
            "AHM001        72427870           0       0.0  4407886140            60.859   \n",
            "KOL001        98267080           0       0.0  4787656160            48.721   \n",
            "PUN001        63478450           0       0.0  3741098180            58.935   \n",
            "DEL001        83451300           0       0.0  3865954470            46.326   \n",
            "HYD001        77617410           0       0.0  3452672510            44.483   \n",
            "\n",
            "           score  \n",
            "branch_id         \n",
            "CHN001     1.000  \n",
            "AHM001     0.793  \n",
            "KOL001     0.678  \n",
            "PUN001     0.608  \n",
            "DEL001     0.426  \n",
            "HYD001     0.300  \n"
          ]
        }
      ]
    },
    {
      "cell_type": "code",
      "source": [
        "print(\"INVENTORY:\", inv.columns.tolist())\n",
        "print(\"SALES:\", sales.columns.tolist())\n",
        "print(\"LEDGER:\", led.columns.tolist())"
      ],
      "metadata": {
        "colab": {
          "base_uri": "https://localhost:8080/"
        },
        "id": "RtxWl0TUupXa",
        "outputId": "8c3a9f62-270f-4e87-8409-685755b1116b"
      },
      "execution_count": 15,
      "outputs": [
        {
          "output_type": "stream",
          "name": "stdout",
```

```
          "text": [
            "INVENTORY: ['product_id', 'branch_id', 'opening_stock', 'reorder_level', 'safety_stock',
'max_stock', 'current_stock', 'warehouse_bin', 'unit_cost', 'stock_value', 'last_movement',
'days_since_last_movement', 'dead_value']\n",
            "SALES: ['so_id', 'line_number', 'product_id', 'quantity', 'unit_price', 'gst_rate', 'line_total',
 'gst_amount', 'line_grand_total', 'customer_id', 'branch_id', 'order_date', 'delivery_date', 'order_status',
'payment_terms', 'total_order_value', 'total_gst_amount', 'grand_total', 'sales_channel', 'customer_type',
'industry_segment', 'category', 'brand', 'region', 'line_revenue', 'order_month']\n",
            "LEDGER: ['movement_id', 'product_id', 'branch_id', 'movement_type', 'movement_date', 'quantity',
'reference_type', 'reference_id', 'running_balance', 'month']\n"
          ]
        }
      ]
    },
    {
      "cell_type": "code",
      "source": [
        "print(inv.columns.tolist())"
      ],
      "metadata": {
        "colab": {
          "base_uri": "https://localhost:8080/"
        },
        "id": "fJr1tVjPupJb",
        "outputId": "67dedeab-a807-4923-f21c-18f2d15c6f96"
      },
      "execution_count": 16,
      "outputs": [
        {
          "output_type": "stream",
          "name": "stdout",
          "text": [
            "['product_id', 'branch_id', 'opening_stock', 'reorder_level', 'safety_stock', 'max_stock',
'current_stock', 'warehouse_bin', 'unit_cost', 'stock_value', 'last_movement', 'days_since_last_movement',
'dead_value']\n"
          ]
        }
      ]
    },
    {
      "cell_type": "code",
      "source": [
        "print(sales.columns.tolist())"
      ],
      "metadata": {
        "colab": {
          "base_uri": "https://localhost:8080/"
        },
        "id": "HPta_wac1CMX",
        "outputId": "f7dd4780-e8f0-420b-e33d-c2e04d190ff6"
      },
      "execution_count": 17,
      "outputs": [
        {
          "output_type": "stream",
          "name": "stdout",
          "text": [
            "['so_id', 'line_number', 'product_id', 'quantity', 'unit_price', 'gst_rate', 'line_total',
'gst_amount', 'line_grand_total', 'customer_id', 'branch_id', 'order_date', 'delivery_date', 'order_status',
'payment_terms', 'total_order_value', 'total_gst_amount', 'grand_total', 'sales_channel', 'customer_type',
'industry_segment', 'category', 'brand', 'region', 'line_revenue', 'order_month']\n"
          ]
        }
      ]
    },
    {
      "cell_type": "code",
      "source": [
        "print(led.columns.tolist())"
      ],
      "metadata": {
        "colab": {
          "base_uri": "https://localhost:8080/"
```

```
        },
        "id": "7xW6VkO11Fzo",
        "outputId": "73b65111-71e2-4e1b-9380-ce519321fc66"
      },
      "execution_count": 18,
      "outputs": [
        {
          "output_type": "stream",
          "name": "stdout",
          "text": [
            "['movement_id', 'product_id', 'branch_id', 'movement_type', 'movement_date', 'quantity',
'reference_type', 'reference_id', 'running_balance', 'month']\n"
          ]
        }
      ]
    },
    {
      "cell_type": "code",
      "source": [],
      "metadata": {
        "id": "Kli3zGfm1QEm"
      },
      "execution_count": 18,
      "outputs": []
    }
  ]
}
```